CSC373H1 - Assignment 1

Question 3

a) The algorithm increments in employees and assigns the first available task with skill demand $\leq$ the current employee's skill, if it exists. If not, the algorithm moves to the next employee.

Algorithm 1 Task Assignment

```
Sort the employees in a non-decreasing order of their skill level so that s_1 ≤ ... ≤ s_n
Sort the tasks in a non-decreasing order of their minimum skill requirement so that d_1 ≤ ... ≤ d_m
j ← 1                                                    ▷ Increments the tasks
for Employee i = 1, ..., n do
    if d_j ≤ s_i then
        Assign task j to employee i
        j ← j + 1
    end if
    if j > m then                                        ▷ No more tasks left
        Break the for loop
    end if
end for
```

b) The worst-case runtime is $O(n \log n + m \log m)$. Sorting the employees takes $O(n \log n)$ time, while sorting the tasks takes $O(m \log m)$ time. For each of the n employees, a constant amount of work is done.

c) Define G_i and O_i to be the sets of tasks assigned to employees $1, \ldots, i$ by the greedy (GRD) and optimal (OPT) approaches respectively. Since GRD considers the employees in order, assume WLOG that OPT also does this (see Lemma). We proceed by induction to show that $\forall i \in \{1, \ldots, n\}, |G_i| = |O_i|$, where $|\cdot|$ denotes set size. This implies $|G_n| = |O_n|$, so GRD is as optimal as OPT.

Base case ($i = 1$): Assume both OPT and GRD has not considered any employee after 1 at this point. Consider two cases:

- Case ($\exists$ task j s.t. $d_j \leq s_1$): OPT assigns employee 1 to such a task. For GRD, this case implies that $d_1 \leq s_1$, so it assigns employee 1 to task 1. Thus, $|G_i| = |O_i| = 1$.
- Case ($\forall$ tasks j, $d_j > s_1$): Both OPT and GRD do not assign employee 1 to a task, so $|G_i| = |O_i| = 0$.

Induction step ($i > 1$): Assume $|O_{i-1}| = |G_{i-1}|$ and both OPT and GRD has not considered any employee after i at this point. Suppose for contradiction that $|O_i| > |G_i|$, which means O_i includes a task j assigned to employee i, but that G_i does not assign employee i to any task. Define $A_i = \{\text{tasks } j : d_j \leq s_i\} \setminus O_{i-1}$ to be the set of available tasks with demands $\leq s_i$, and similarly $B_i = \{\text{tasks } j : d_j \leq s_i\} \setminus G_{i-1}$. Task $j \in A_i$ but $\notin B_i$ since GRD must have assigned it to some other employee before employee i. Note that $|A_i| = |B_i|$ since $|O_{i-1}| = |G_{i-1}|$ and both OPT and GRD has not considered any employee after i. This means B_i must contain an additional available

task with demand $\leq s_i$ for the sets to be equal in size. Then, GRD can just assign employee i to this task, yielding a contradiction. We conclude $|O_i| = |G_i|$, as needed.

Lemma: Define OPT as the optimal approach that considers the employee in order and A as another approach that does not follow this order but is otherwise as similar to OPT as possible. We show OPT is at least as optimal as A. Since A does not follow the order, there must be ≥ 1 pair of employees x, y s.t. it considers y before considering x and $x < y$. Note $x < y \implies s_x \leq s_y$.

- If A assigns y to task m and x to task n, this implies either
  - d_m, d_n both $\leq s_x, s_y$, in which case OPT might make the same or opposite assignments depending on the order of m and n, or
  - $d_n \leq s_x$ and $d_m \in (s_x, s_y]$, in which case OPT would make the same assignments.
- If A assigns y to task m and leaves x unassigned, this implies either
  - $d_m \leq s_x$, in which case OPT would assign x to m and leave y unassigned instead, or
  - $d_m \in (s_x, s_y]$, in which case OPT would make the same choice.

  Notice however that for the first sub-case, if there is an available task n s.t. $s_x < d_n \leq s_y$, OPT would also assign y to n, meaning it has 1 more task assigned than A.
- If A leaves both x, y unassigned, this implies there are no available tasks m s.t. $d_m \leq s_y$, in which case OPT would make the same choice.

In all cases, the number of tasks assigned to each pair of employees by A is $\leq$ that of OPT.

Question 4

a) After each iteration d, the corresponding project d should not have any employee who feels inferior since the algorithm should have moved all such employees to project $d + 1$. Since the algorithm stops when the inferiority-freeness constraint holds for a newly-created project k, it must hold for all previously created projects as well.

b) Base case: Suppose $k = 2$. Since i_2 feels inferior to i_1, any solution must assign i_2 to project 2 while keeping i_1 in project 1.

Inductive step: Let $k > 2$, and assume any solution assigns $(i_1, \ldots, i_{k-1})$ to $k - 1$ different projects since $s_1 > \ldots > s_{k-1}$ and $t_1 < \ldots < t_{k-1}$. Note that i_{k-1} feels inferior to i_k implies any employee before i_{k-1} must also feel inferior to i_k, since $s_1 > \ldots > s_{k-1} > s_k$ and $t_1 < \ldots < t_{k-1} < t_k$. Thus, i_k cannot be assigned to the same project as any other employee before it. This means any solution must assign i_k to a new project k, and so any solution must assign $(i_1, \ldots, i_k)$ to k different projects.

c) Consider an arbitrary set of employees, and assume it is sorted as in part d. If it has ≥ 1 sequence s.t. each employee feels inferior to the previous, it must have a longest such sequence. (Notice the sequence does not have to be contiguous.) Define this longest sequence as $S = (i_1, \ldots, i_k)$ for some $k \leq n$. By part b, any solution must have at least k projects. Notice that the greedy solution (GRD) uses exactly k projects for this set. This is because for every employee:

- By the Lemma, the employee does not belong to S $\implies$ there must exist ≥ 1 employee in S where neither employee feels inferior to the other. So, GRD can assign it to the same project as ≥ 1 employee in S without it or the other feeling inferior. This does not affect the overall number of projects.
- If GRD cannot assign the employee in this manner, by the contrapositive of the Lemma, it must belong to S. GRD, which produces a feasible solution by part a, already splits S up between k projects by part b.

We conclude that # of projects needed by GRD $= k \leq$ # of projects needed by any solution.

Lemma: We show that an employee $i \notin S \implies \exists j \in S$ where neither i nor j feels inferior to the other. Suppose for contradiction that $i \notin S$ but $\forall j \in S$, either i feels inferior to j or vice versa.

- If all j's feel inferior to i or i feels inferior to all j's, i must be at the beginning or end of S by definition, a contradiction.
- If $\exists j \in S, s_j < s_i \wedge t_i < t_j$ and $\exists j' \in S, s_{j'} > s_i \wedge t_i > t_{j'}$, then $s_j < s_i < s_{j'} \wedge t_{j'} < t_i < t_j$, and i must be nested within S by definition, a contradiction.
- If the previous cases cannot be reached, then there must exist an employee $j \in S$ where neither i nor j feels inferior to the other, a contradiction.

d) For the algorithm, since it sorts the employee twice, it only needs to compare each adjacent pair of employees.

Algorithm 2 Project Division

```
Sort the employees in order of their start time so that s_1 ≤ ... ≤ s_n
If there are employees sharing a start time, sort them in non-decreasing order of their finish time
Initialize an empty set Project 1 as a linked list and add the employees in order
CurrentProject ← Project 1
InferiorityExists ← True
d ← 2
while InferiorityExists is True do
    Initialize an empty set Project d as a linked list
    PreviousStart ← start time of first employee in CurrentProject
    PreviousEnd ← finish time of first employee in CurrentProject
    for Employee e_i in CurrentProject do                ▷ Iterating through the linked list
        if PreviousStart < s_i and PreviousEnd > t_i then
            Add {e_i} to Project d
            Remove {e_i} from CurrentProject
        else
            PreviousStart ← s_i
            PreviousEnd ← t_i
        end if
    end for
    if Project d is ∅ then          ▷ Only happens when CurrentProject is inferiority-free
        InferiorityExists ← False                                          ▷ Breaks the while loop
    else
        CurrentProject ← Project d
        d ← d + 1
    end if
end while
```

The algorithm runs in $O(n^2)$ time since there can be at most n projects and n iterations in the while loop. For each project, the algorithm loops over all $\leq n$ employees assigned to it and performs a constant amount of work. Also, sorting the employees takes $O(n \log n)$ time, and moving employees from one project to another takes $O(n)$ time overall due to the linked list implementation.

CSC373H1 - Assignment 2

Question 4

a) The network instance has a source S and sink T. It has a node (i, t) for each city $i \in V$ and for each $t \in \{0, \ldots, D\}$, which represent the cities at different times. Consider each t to be a layer.

For each edge $i \to j \in E$ and $t \in \{0, \ldots, D-1\}$, there is an edge between nodes (i, t) and $(j, t+1)$ with capacity $c_{i,j}$. Also, for each city $i \in V$ and $t \in \{0, \ldots, D-1\}$, there is an edge between (i, t) and $(i, t+1)$ with capacity P.

Connect S to $(a, 0)$ and T to (b, D), both with a capacity of P. For each $i \in V \setminus \{b\}$, connect T to

(i, D) with capacity 0. This is to ensure that Ford-Fulkerson doesn't send any flow through these nodes to T.

If the amount of flow into T is P, the plan is feasible within time D, and infeasible otherwise.

b) We use the function BinSearch, which will call Helper. Helper attempts to find a solution with a given flow using Ford-Fulkerson; it returns the graph if true, and false otherwise. In BinSearch, we repeat calls to helper with input sizes 2^k for k from 1 to some i such that 2^i is the first power of $2 \geq D'$, the smallest feasible D. Then, we perform a binary search on the interval $(2^{i-1}, 2^i] = (2^{\lceil \log D' \rceil - 1}, 2^{\lceil \log D' \rceil}]$ since D' must lie in it. For each iteration of the search, Helper creates a new graph and checks if a flow of P is feasible.

Algorithm 3 Binary search for the smallest feasible D

```
1:  function BinSearch(G, P)
2:      i = 1
3:      while Helper(G, P, 2^i) is Null do                    ▷ Helper is on next page
4:          i ← i + 1
5:      end while
6:      if i = 1 then
7:          Return 1
8:      end if
9:      low ← 2^(i−1)
10:     high ← 2^i
11:     while low ≠ high - 1 do
12:         mid ← (low + high) / 2
13:         if Helper(G, P, mid) is not Null then
14:             high ← mid
15:         else
16:             low ← mid
17:         end if
18:     end while
19: return high        ▷ High is always the smallest D upon termination. This is proved below.
20: end function
```

We show that BinSearch finds D'. For the second while loop, the precondition is that $i > 1$ since $i = 1$ is addressed in the if statement on line 6. $i > 1 \implies 2^i > 2^{i-1} + 1$, so the loop runs at least once. We prove its correctness as follows:

Loop invariant for iteration k of the second while loop (LI_k): high is feasible (i.e., Helper(high) is not null) $\wedge$ low is not feasible $\wedge$ high $-$ low is a power of 2.

- Base case: LI_0 holds since by the first while loop, Helper(2^i) is not null and Helper(2^{i-1}) is null. $2^i - 2^{i-1} = 2^{i-1}$, which is a power of 2.
- Assume LI_k and consider iteration $k+1$. If Helper(mid) is not null, high = mid is feasible, while low remains infeasible by LI_k. If Helper(mid) is null, low = mid is infeasible, while high remains feasible by LI_k. Also, high $-$ mid = high $- \frac{\text{high}+\text{low}}{2} = \frac{\text{high}-\text{low}}{2}$, and similarly

for $\text{mid} - \text{low}$. Since $\text{high} - \text{low}$ is a power of 2 by LI_k, $\frac{\text{high}-\text{low}}{2}$ is also. Thus, LI_{k+1} holds.

The second while loop terminates since by LI, the size of (low, high] is a power of 2, and it is halved in each iteration. This means (low, high] will eventually have a size of 1, which corresponds to $\text{low} = \text{high} - 1$, the termination requirement. When the loop terminates, by the LI, high is feasible and $\text{low} = \text{high} - 1$ is infeasible, meaning that $\text{high} = D'$ is returned, as needed.

Algorithm 4 Helper function to create the network

```
function HELPER(G, P, D)
    Create S, T
    for t = 0, ..., D do
        for i ∈ V do
            Create node (i, t)
            if t > 0 then
                Create edge (i, t − 1) → (i, t) with capacity P
            end if
        end for
    end for
    for (i, j) ∈ E do
        for t = 0, ..., D − 1 do
            Create edge (i, t) → (j, t + 1) with capacity c_{i,j}
        end for
    end for
    for i ∈ V do
        Create edge (i, D) → T with capacity 0
    end for
    Change edge (b, D) → T to capacity P
    Create edge S → (a, 0) with capacity P
    Compute a maximum flow f in the above instance with Ford-Fulkerson
    if f = P then
        return the constructed network
    else
        return Null
    end if
end function
```

We show that a maximum flow of P is equivalent to a feasible D.

($\implies$) Take any flow P, which is the maximum flow that S can send out and T can receive. We want to prove that D is feasible. Since the only edge to T with non-zero capacity is $(b, D) \to T$, it must be saturated with flow P. By conservation of flow, (b, D) must be receiving P flow for it to send it to T. This means every part of this flow passes through $(a, 0)$ and (b, D) along some $s - t$ path, which is length D if disregarding S and T since edges are only between nodes at different t. This means P people can travel to b in D steps of 1 unit time each, meaning D is a feasible deadline.

($\Longleftarrow$) Assume D is feasible. We want to show that there is a valid flow P in the network described, which is the maximum possible. Notice:

- Since the only edge to T with non-zero capacity is $(b, D) \to T$, all flow must eventually arrive at (b, D), which itself sends P flow.
- All edge capacities are the same as in G. Edges between the same cities at different times represent people who are waiting and have capacity P (each city can hold P people).
- The length of any path from $(a, 0)$ to (b, D) is D since edges are only between nodes at different t (i.e., edges are always forward).

By the above and since D is feasible, a flow P sent from S can take the same actions as in the original schedule, moving in a forward edge of the network for each increment of time, and arrive at (b, D). Moreover, conservation of flow is respected since in the original schedule, the number of people going into or staying at a city at some t is the same as the number leaving or staying at $t + 1$. Thus, there is a valid and maximum flow of P in the network.

c) Define $n = |V|, m = |E|$. Note that every graph G has $\geq$ one $a - b$ path and its edge capacities are all ≥ 1. We can send each person into the graph individually with no spaces in between, like a queue. In this way, the last person must be free to move out of node a at time $P + 1$ after all preceding people have left a. Since this last person need only travel for $\leq n - 1$ nodes before reaching b, which would take time $n - 1$, every graph G needs at most $P + 1 + n - 1 = P + n$ time for all people to arrive at b. Thus, $D' \leq P + n$.

The overall runtime of BinSearch is $O((\log D')(n + m)D'P) = O\Big((\log(P + n))(n + m)(P + n)P\Big)$, since:

- The first while loop iterates over powers of 2 and stops at the smallest such power $\geq D'$, meaning that it runs for $\lceil \log D' \rceil \in O(\log D')$ steps. Each call to Helper calls Ford-Fulkerson, which is $O((n + m)D'P)$ since the maximum capacity is P and there are $2^{\lceil \log D' \rceil} \approx D'$ layers of nodes and edges. Also, the loops at lines 3, 11, and 16 in Helper are $O(D'n)$, $O(D'm)$, and $O(n)$ respectively. In total, the first while loop of BinSearch is in $O((\log D')(n + m)D'P)$.
- The second while loop performs binary search over the space $(2^{\lceil \log D' \rceil - 1}, 2^{\lceil \log D' \rceil}]$, which has size $2^{\lceil \log D' \rceil - 1}$, meaning that it runs for around $O(\log(2^{\lceil \log D' \rceil - 1})) = O(\log D')$ iterations. Each iteration of the second while loop runs Helper, which has the same runtime as noted above. In total, the second while loop of BinSearch is $O((\log D')(D'n + D'm)P)$.
- All other operations are $O(1)$.

CSC373H1 - Assignment 3

Question 2

a) Define $w_i = 1$ if worker i is hired and 0 otherwise. The total cost is then $\sum_{i=1}^{n} c_i w_i$ and the

program is

$$\begin{aligned}
\text{minimize} \quad & \sum_{i=1}^{n} c_i w_i \\
\text{s.t.} \quad & \sum_{i: s_i \le a < t_i} w_i \ge 1 \\
& \sum_{i: s_i \le t_j < t_i} w_i \ge 1, \forall j \text{ s.t. } a < t_j < b \\
& w_i \in \{0, 1\}, \forall i = 1, \ldots, n.
\end{aligned}$$

The first constraint enforces that at least 1 worker who delivers the food from position a is hired. The second constraint enforces that for every worker j who stops at/after a but before reaching b, regardless of whether they are hired or not, we must hire ≥ 1 other worker i who starts at/before t_j and who goes even closer towards b. The third constraint enforces that w_i is binary and non-negative.

On top of constraints 1 and 3, each worker has at most $n-1$ versions of constraint 2, one for every other worker. This means $\le n+1$ constraints in total for each worker and $O(n^2)$ constraints overall, which is polynomial for n workers.

b) We prove a one-to-one correspondence between solutions of the original problem and the linear program. By proving this, we conclude that any minimum value obtained by the program must be the optimal solution for the problem.

Take any (not necessarily optimal) solution to the original problem. By assumption, there is a set of hired workers S s.t. $[a, b] \subseteq \bigcup_{i \in S} [s_i, t_i]$ (since they can transport the food from a to b). Set $w_i = 1$ for $i \in S$ and $w_i = 0$ for $i \notin S$. Note there must be a worker j who delivers the food out of a, i.e. $a \in [s_j, t_j)$, so constraint 1 is met. Now, for any worker j s.t. $t_j \in (a, b)$, hired or not, it must be that $t_j \in [s_i, t_i)$ for another hired worker i. If this were not so, t_j would lie in a subset of $[a, b]$ not covered by the stretch of any hired worker, i.e. $t_j \notin \bigcup_{i \in S} [s_i, t_i]$, which implies $t_j \notin (a, b)$ (a contradiction). Then, constraint 2 is met, so any feasible solution of the original problem can be converted into an instance of the linear program.

Take any instance of the linear program. By constraint 1, we hire a worker j who can deliver the food out of a, i.e. $w_j = 1$ and $a \in [s_j, t_j)$. For every worker j with $t_j \in (a, b)$, we hire ≥ 1 other worker who could pick up the food at t_j and deliver it to a point closer to b. This other worker must exist since constraint 2 guarantees a worker i s.t. $w_i = 1$ and $s_i \le t_j < t_i$ for such a j. Thus, we are able to hire a subset of workers who can deliver the food from a to b since

- The first hired worker who delivers the food out of a is counted as a j in constraint 2, so another hired worker will pick the food up from them.
- For the last j in constraint 2, i.e. the j with the latest $t_j \in (a, b)$, we hire another worker i who picks the food up from them. Since $s_i \le t_j < t_i$ and t_j is the latest $t \in (a, b)$, $t_i \notin (a, b)$, meaning $s_i < b \le t_i$. Thus, i can deliver the food to b, as required.
- Every hired worker with t in between the previous two workers satisfy constraint 2, so there will always be another worker who can pick the food up from them.

Then, there is a one-to-one correspondence between any instance of the linear program and feasible solution of the original problem. This means that minimizing the linear program yields the optimal solution for the problem.

CSC373H1 - Assignment 4

Question 1

a) For any $c \geq 1$, fix $B > c + c^2$. Suppose there are only proposals 1 and 2, where $x_1 = c, y_1 = 1 + c$ and $x_2, y_2 = B$. Only one proposal can be picked by any algorithm since $\sum_{i \in \{1,2\}} x_i = B + c > B$. ALG picks proposal 1 since $\frac{y_1}{x_1} = \frac{1+c}{c} > \frac{y_2}{x_2} = \frac{B}{B} = 1$. OPT picks proposal 2 since $y_2 = B > c + c^2 \geq y_1 = 1 + c$. Profit(ALG) $= 1 + c < \frac{B}{c} = \frac{1}{c}$ Profit(OPT) since $B > c + c^2$, as desired.

b) Assume that $k < n$, so project $k+1$ exists. Define $\epsilon = \frac{B - \sum_{i=1}^{k} x_i}{x_{k+1}}$, which $\in [0, 1)$ since $\sum_{i=1}^{k} x_i + x_{k+1} > B$. Define OPT as the optimal solution with only an integral number of projects. Define OPT' as the solution which extends OPT by spending its remaining budget to pick a fraction of a project.

Consider a solution S that sorts the projects as in part (a) and picks $\{1, \ldots, k\}$ and an ϵ-fraction of $k+1$, s.t. $\sum_{i=1}^{k} x_i + \epsilon x_{k+1} = B$. By Lemmas 1 and 2, this is the optimal solution if B is to be spent entirely. Then, Profit(S) $\geq$ Profit(OPT') $\geq$ Profit(OPT). From this, $2 \max(\sum_{i=1}^{k} y_i, y_{k+1}) \geq \sum_{i=1}^{k} y_i + y_{k+1} > \sum_{i=1}^{k} y_i + \epsilon y_{k+1} =$ Profit(S) $\geq$ Profit(OPT). Since the modified algorithm picks $\max(\sum_{i=1}^{k} y_i,\ y_{k+1})$, it achieves a 2-approximation.

Lemma 1: Replacing any subset of the projects chosen by S with later projects cannot increase the total profit. Assume $\{1, \ldots, n\}$ are sorted as in part (a). Define $A \subseteq \{1, \ldots, k+1\}$ as the set of projects to be replaced, $C \subseteq \{k+2, \ldots, n\}$ as their replacement, and $\alpha_j \in (0, 1]$ as the fraction of $j \in C$ to include. Note that

- $\forall i \in A, \dfrac{y_i}{x_i} \geq \dfrac{y_{k+1}}{x_{k+1}}$, so $\sum_{i \in A} y_i \geq \dfrac{y_{k+1}}{x_{k+1}} \sum_{i \in A} x_i$, and
- $\forall j \in C, \alpha_j \dfrac{y_j}{x_j} \leq \alpha_j \dfrac{y_{k+1}}{x_{k+1}}$, so $\sum_{j \in C} \alpha_j y_j \leq \dfrac{y_{k+1}}{x_{k+1}} \sum_{j \in C} \alpha_j x_j$.

If $k + 1 \notin A$:

- $\sum_{j \in C} \alpha_j x_j \leq \sum_{i \in A} x_i$, or else the projects in C would not fit in the spaces in the budget left by removing A.
- Thus, $\sum_{j \in C} \alpha_j y_j \leq \dfrac{y_{k+1}}{x_{k+1}} \sum_{j \in C} \alpha_j x_j \leq \dfrac{y_{k+1}}{x_{k+1}} \sum_{i \in A} x_i \leq \sum_{i \in A} y_i$, which is the total profit of A.

If $k + 1 \in A$:

- $\sum_{j \in C} \alpha_j x_j \leq \sum_{i \in A \setminus \{k+1\}} x_i + \epsilon x_{k+1}$, or else the projects in C would not fit in the spaces in the budget left by removing A.
- Thus, $\sum_{j \in C} \alpha_j y_j \leq \dfrac{y_{k+1}}{x_{k+1}} \sum_{j \in C} \alpha_j x_j \leq \dfrac{y_{k+1}}{x_{k+1}} (\sum_{i \in A \setminus \{k+1\}} x_i + \epsilon x_{k+1}) \leq \sum_{i \in A \setminus \{k+1\}} y_i +$

$\frac{y_{k+1}}{x_{k+1}}(\epsilon x_{k+1}) = \sum_{i \in A \setminus \{k+1\}} y_i + \epsilon y_{k+1}$, which is the total profit of A.

Lemma 2: Define $\alpha = \frac{B - \sum_{i \in \{1,\ldots,k+1\} \setminus \{j\}} x_i}{x_j}$ for $j < k+1$. If S instead picks the entire project $k+1$ and only picks an α-fraction of project j, the total profit cannot increase. This is since

$$\begin{aligned}
\frac{y_j}{x_j} \geq \frac{y_{k+1}}{x_{k+1}} &\iff \frac{\sum_{i=1}^{k+1} x_i - B}{x_j} y_j \geq \frac{\sum_{i=1}^{k+1} x_i - B}{x_{k+1}} y_{k+1} \\
&\iff (1 - \frac{B - \sum_{i \in \{1,\ldots,k+1\} \setminus \{j\}} x_i}{x_j}) y_j \geq (1 - \frac{B - \sum_{i=1}^{k} x_i}{x_{k+1}}) y_{k+1} \\
&\iff y_j + \epsilon y_{k+1} \geq \alpha y_j + y_{k+1}.
\end{aligned}$$

CSC373H1 - Ethics Module Assignment

Construct the network with the set of nodes $\{s, t, v_1, \ldots, v_n, w_1, \ldots, w_m\}$, where each v_i represents a country and each w_r represents a warehouse, with the following edges:

- (s, w_r) with capacity s_r for each w_r
- (v_i, t) with capacity c_i for each v_i
- (w_r, v_i) with capacity s_r if (i, r) is in the list of pairs and 0 otherwise

Then, the Ford-Fulkerson algorithm computes an integral maximal flow. We prove that there is a one-to-one correspondence between the flow of the network (not necessarily maximal) and total number of vaccines that can be allocated:

- (Flow of f in the network $\implies$ f vaccines can be allocated) Assume there is a flow f in the network. This means that f moves from s into the (w_r, v_i) edges with positive capacities and then into t. Since each w_r can receive at most s_r flow from s and each v_i can send at most c_i flow, each (w_r, v_i) edge cannot have more than $\min(s_r, c_i)$ flow. Equivalently, since each warehouse can supply up to s_j vaccines and each country can receive up to c_i vaccines, it must be that each warehouse j cannot send more than $\min(s_j, c_i)$ vaccines to a given country i. Additionally, if $(i, r) \notin$ the list of pairs, (w_r, s_i) cannot send flow, which means warehouse r does not send any vaccines to country i. Since the flow f meets all constraints in the original problem, f vaccines can therefore be allocated in total.
- (f vaccines can be allocated $\implies$ flow of f in the network) Assume that f vaccines can be allocated in total. Now, since each warehouse r can supply up to s_r doses in total and each country i is willing to purchase up to c_i doses, each warehouse r cannot send more than $\min(s_r, c_i)$ vaccines to a given country i. In addition, each warehouse r can only send vaccines to a country i if $(i, r) \in$ the list of pairs. In the constructed network, this is exactly equivalent to how each w_r can receive at most s_r flow from s and each v_i can send at most c_i flow to t, meaning each (w_r, v_i) edge cannot carry a flow of more than $\min(s_r, c_i)$. Moreover, (w_r, v_i) has positive flow only if $(i, r) \in$ the list of pairs. Thus, a flow of f is feasible in the network.

This proves the one-to-one correspondence, meaning that maximizing the flow of the network is equivalent to maximizing the total number of vaccines that can be allocated, as desired.